

There are some exceptions where short variable names are fine. For example, it's common to use `i` (for *index*) as a variable name in `for` loops that loop over a range of numbers or indexes of a list, and `j` and `k` (because they come after *i* in the alphabet) if you have nested loops:

```
>>> for i in range(10):
...     for j in range(3):
...         print(i, j)
...
0 0
0 1
0 2
1 0
--snip--
```

Another exception is using `x` and `y` for Cartesian coordinates. In most other cases, I caution against using single-letter variable names. Although it might be tempting to use, say, `w` and `h` as shorthand for `width` and `height`, or `n` as shorthand for `number`, these meanings might not be apparent to others.

DN'T DRP LTTRS FRM YR SRC CD

Don't drop letters from your source code. Although dropped letters in names like `memcpy` (memory copy) and `strcmp` (string compare) were popular in the C programming language before the 1990s, they're an unreadable style of naming that you shouldn't use today. If a name isn't easily pronounceable, it isn't easily understood.

Additionally, feel free to use short phrases that can make your code read like plain English. For example, `number_of_trials` is more readable than simply `number_trials`.

Too Long Names

In general, the larger the name's scope, the more descriptive it should be. A short name like `payment` is fine for a local variable inside a single, short function. But `payment` might not be descriptive enough if you use it for a global variable across a 10,000-line program, because such a large program might process multiple kinds of payment data. A more descriptive name, such as `salesClientMonthlyPayment` OR `annual_electric_bill_payment`, could be more